

Task 1: [2 mark]

The code provided below is for a **recursive function** that computes the edit distance between two strings x and y. Modify this code such that:

- (a) The function measures a **weighted edit distance**, where
 - insertion cost = 2,
 - deletion cost = 2,
 - substitution cost = 2, if a letter is substituted with another letter or a digit is substituted with another digit
 - substitution cost = 4, if a letter is substituted with a digit or vice versa
- (b) use the [time.time\(\) method](#) to measure the execution time.

Note 1: If you are not familiar with recursive functions, you must first watch the short optional video in the lesson, which reviews this topic. Once you understand how to compute *n!* using a recursive function, you can solve this task quickly.

Note 2: More advanced coders can consider adding [tracing](#) capabilities to their recursive function. This optional task carries no marks but could be an insightful way to understand recursion (do this as part of your group work).

```
import time

#How many insertions, deletions, or substitutions does it take to turn x into y?
def edDistRecursive(x, y):

    # If either x or y is empty, then one or more (depending on their length) INSERTIONS or DELETIONS are needed to transform x to y.
    if len(x) == 0:
        #print(f'\t Converting "{x}" TO "{y}" requires {len(y)} INSERTIONS, therefor it costs {len(y)}')
        return len(y)
    if len(y) == 0:
        #print(f'\t Converting "{x}" TO "{y}" requires {len(x)} DELETIONS, therefor it costs {len(x)}')
        return len(x)

    # When neither x nor y is empty, we must compute the cost of each operation and find the least costly one.
    delta = 1 if x[-1] != y[-1] else 0
    diagonal_or_substitution_cost = edDistRecursive(x[:-1], y[:-1]) + delta #what's the cost of SUBSTITUTING the last character of x with the last character of y
    vertical_or_deletion_cost      = edDistRecursive(x[:-1], y)             + 1    #what's the cost of DELETING the last character of x
    horizontal_or_insertion_cost   = edDistRecursive(x, y[:-1])             + 1    #what's the cost of INSERTING the last character of x into y

    # What's the least costly operation?
    minValue = min(diagonal_or_substitution_cost, vertical_or_deletion_cost, horizontal_or_insertion_cost)
    return minValue
```

Expected Output - demonstrate the functionality of your function by trying the two test cases shown below:

Edit Distance between "intention" and "execution": ??????	Edit Distance between "ABC" and "456": ??????
--- Executed in: 2.647733449935913 seconds ---	--- Executed in: 0.00036716461181640625 seconds ---

Edit distance

Let $D[0, j] = j$, and let $D[i, 0] = i$

Otherwise, let
$$D[i, j] = \min \begin{cases} D[i - 1, j] + 1 \\ D[i, j - 1] + 1 \\ D[i - 1, j - 1] + \delta(x[i - 1], y[j - 1]) \end{cases}$$

$\delta(a, b)$ is 0 if $a = b$, 1 otherwise

A simple recursive algorithm:

```
def edDistRecursive(x, y):
    if len(x) == 0: return len(y)
    if len(y) == 0: return len(x)
    delt = 1 if x[-1] != y[-1] else 0
    diag = edDistRecursive(x[:-1], y[:-1]) + delt
    vert = edDistRecursive(x[:-1], y) + 1
    horz = edDistRecursive(x, y[:-1]) + 1
    return min(diag, vert, horz)
```

← prefixes of x and y currently under consideration

Recursively solve smaller problems

Task 2: [3 marks]

To complete this task, you need to install two libraries:

- Install the First and Last Names Dataset (<https://github.com/philipperemy/name-dataset>). (`!pip install names-dataset`)
- Install the jellyfish library for doing approximate and phonetic matching of strings (<https://github.com/jamesturk/jellyfish>) (`!pip install jellyfish`)

A. Write a function that takes your firstName and lastName as input and checks if they exist in the First and Last Names dataset. If they do, then print out the most likely nationality and gender [1 mark]

Expected Output:

```
Total number of first names: 727556
Total number of last names: 983826

First Name "John" found: {'country': {'CA': 0.036, 'CO': 0.054, 'FR': 0.028, 'GB': 0.233,
The Most likely nationality: US
The Most likely gender: M

Last Name "Jones" found: {'country': {'CA': 0.022, 'ES': 0.004, 'FR': 0.009, 'GB': 0.406,
The Most likely nationality: US
```

B. Write a function that takes your firstName and lastName as input and prints out their Soundex codes (use the Soundex function in the jellyfish library). [0.5 mark]

Expected Output:

```
Soundex code for "John" is: J500
Soundex code for "Jones" is: J520
```

C. Write a function that takes your firstName as input and prints out a list of first names in the dataset whose Soundex codes are the same as your firstName's Soundex code. [1 mark]

D. Write a function that takes your firstName and lastName as input and returns their Levenshtein Distance (use the Levenshtein Distance function in the jellyfish library). [0.5 mark]

Task 3: [2.5 marks]

The code provided [HERE](#) is for a function that computes the edit distance between two strings x and y , using dynamic programming. Modify this code such that:

- (a) the function measures the **Levenshtein distance**, i.e. insertion cost = 1, deletion cost = 1, substitution cost = 2
- (b) use `%time` or `time.time() method` to measure the execution time.
- (c) Augment the code such that it outputs a DP table including backtrace pointers in each cell (as shown in the sample output below).

Note1: Please see [Chapter 2, section 2.8](#) for more details on The Minimum Edit Distance Algorithm and backtrace.

Note2: the direction of vertical arrows (up/down) depends on the orientation of the DP table (e.g. see the two sample tables on the right), therefore it could be easier if you think of possible operations as:

- 1. **Substitution OR no change (diagonal):** Cost (↖ substitution) = 2 OR Cost (↖ no-change) = 0
- 2. **Deletion (vertical: down/up):** Cost(↑↓ deletion) = 1
- 3. **Insertion (horizontal):** Cost(← insertion) = 1

Substitute D[i-1 , j-1]	Delete D[i-1 , j]
Insert D[i , j-1]	← ↖ ↑

Expected Output: (fill in place of question marks)

Completed DP table after all subproblems are solved

	#	e	x	e	c	u	t	i	o	n
#	0↑	1←	2←	3←	4←	5←	6←	7←	8←	
i	1↑	?	?	?	?	?	?	?	?	
n	2↑	?	?	?	?	?	?	?	?	
t	3↑	?	?	?	?	?	?	?	?	
e	4↑	?	?	?	?	?	?	?	?	
n	5↑	?	?	?	?	?	?	?	?	
t	6↑	?	?	?	?	?	?	?	?	
i	7↑	?	?	?	?	?	?	?	?	
o	8↑	?	?	?	?	?	?	?	?	

Dan Jurafsky



Adding Backtrace to Minimum Edit Distance

Base conditions:
 $D(i,0) = i$
 $D(0,j) = j$

Termination:
 $D(N,M)$ is distance

Recurrence Relation:
For each $i = 1 \dots M$
For each $j = 1 \dots N$
$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 & \text{deletion} \\ D(i,j-1) + 1 & \text{insertion} \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases} & \text{substitution} \end{cases}$$

$$ptr(i,j) = \begin{cases} \text{LEFT} & \text{insertion} \\ \text{DOWN} & \text{deletion} \\ \text{DIAG} & \text{substitution} \end{cases}$$

n	9	↓ 8	↖↖ 9	↖↖ 10	↖↖ 11	↖↖ 12	↓ 11	↓ 10	↓ 9	↖ 8
o	8	↓ 7	↖↖ 8	↖↖ 9	↖↖ 10	↖↖ 11	↓ 10	↓ 9	↖ 8	← 9
i	7	↓ 6	↖↖ 7	↖↖ 8	↖↖ 9	↖↖ 10	↓ 9	↖ 8	← 9	← 10
t	6	↓ 5	↖↖ 6	↖↖ 7	↖↖ 8	↖↖ 9	↖ 8	← 9	← 10	↖ 11
n	5	↓ 4	↖↖ 5	↖↖ 6	↖↖ 7	↖↖ 8	↖↖ 9	↖↖ 10	↖↖ 11	↖ 10
e	4	↖ 3	← 4	↖ 5	← 6	← 7	↖ 8	↖↖ 9	↖↖ 10	↓ 9
t	3	↖↖ 4	↖↖ 5	↖↖ 6	↖↖ 7	↖↖ 8	↖ 7	← 8	↖↖ 9	↓ 8
n	2	↖↖ 3	↖↖ 4	↖↖ 5	↖↖ 6	↖↖ 7	↖↖ 8	↓ 7	↖↖ 8	↖ 7
i	1	↖↖ 2	↖↖ 3	↖↖ 4	↖↖ 5	↖↖ 6	↖↖ 7	↖ 6	← 7	← 8
#	0	1	2	3	4	5	6	7	8	9
	#	e	x	e	c	u	t	i	o	n

Source = intention, Target = execution

	#	e	x	e	c	u	t	i	o	n
#	0	← 1	← 2	← 3	← 4	← 5	← 6	← 7	← 8	← 9
i	↑ 1	↖↖ 2	↖↖ 3	↖↖ 4	↖↖ 5	↖↖ 6	↖↖ 7	↖ 6	← 7	← 8
n	↑ 2	↖↖ 3	↖↖ 4	↖↖ 5	↖↖ 6	↖↖ 7	↖↖ 8	↑ 7	↖↖ 8	↖ 7
t	↑ 3	↖↖ 4	↖↖ 5	↖↖ 6	↖↖ 7	↖↖ 8	↖ 7	← 8	↖↖ 9	↑ 8
e	↑ 4	↖ 3	← 4	↖ 5	← 6	← 7	← 8	↖↖ 9	↖↖ 10	↑ 9
n	↑ 5	↑ 4	↖↖ 5	↖↖ 6	↖↖ 7	↖↖ 8	↖↖ 9	↖↖ 10	↖↖ 11	↖ 10
t	↑ 6	↑ 5	↖↖ 6	↖↖ 7	↖↖ 8	↖↖ 9	↖ 8	← 9	← 10	↖ 11
i	↑ 7	↑ 6	↖↖ 7	↖↖ 8	↖↖ 9	↖↖ 10	↑ 9	↖ 8	← 9	← 10
o	↑ 8	↑ 7	↖↖ 8	↖↖ 9	↖↖ 10	↖↖ 11	↑ 10	↑ 9	↖ 8	← 9
n	↑ 9	↑ 8	↖↖ 9	↖↖ 10	↖↖ 11	↖↖ 12	↑ 11	↑ 10	↑ 9	↖ 8

Figure 2.19 When entering a value in each cell, we mark which of the three neighboring cells we came from with up to three arrows. After the table is full we compute an **alignment** (minimum edit path) by using a **backtrace**, starting at the **8** in the lower-right corner and following the arrows back. The sequence of bold cells represents one possible minimum cost alignment between the two strings. Diagram design after [Gusfield \(1997\)](#).

```
?

      n      9↑      ?      ?      ?      ?      ?      ?      ?      ?

?

-----
--
Levenshtein distance between "intention" and "execution": 8
--- Executed in: 0.018010377883911133 seconds ---
```

Task 4: [2.5 marks]

- (a) Copy your solution code for Task3 into a new code cell
- (b) Extend the functionality of your solution for Task3 by implementing a **printBackTrace()** function. The job of this function is to choose one of the optimal (least costly) paths to backtrace, and list all the operations in that path.

Sample* Expected Output:

```
-----
Source: intention , Target: execution

row:-1, col:-1= 8↖ SUB "n" with "n"
row:-2, col:-2= 8↖ SUB "o" with "o"
row:-3, col:-3= 8↖ SUB "i" with "i"
row:-4, col:-4= 8↖ SUB "t" with "t"
row:-5, col:-5= 8←↖↑ SUB "n" with "u"
row:-6, col:-6= 6← INS "c"
row:-6, col:-7= 5←↖ SUB "e" with "e"
row:-7, col:-8= 5←↖↑ SUB "t" with "x"
row:-8, col:-9= 3←↖↑ SUB "n" with "e"
row:-9, col:-10= 1↑ DEL "i"
-----
```

Note*: above sample output shows only one of the optimal paths which could be taken. Your output could differ depending on your choice of operation (SUB, INS, DEL) when two or more operations cost the same.

</END OF ETIVITY>

Teaching team:

- Level of difficulty (easy/fair/hard)
- Task3: use other sample words and remove the results from expected output
- Task4: favour one operation over other(s) when all cost the same, e.g. SUB
- Compute the time complexity of "intention" -> "execution" for both functions (<https://www.youtube.com/watch?v=D6xkbGLQes> to be added)